2 → tipovi

- Ugrađeni vs korisnički tipovi (12)
- Veličina tipske deskriptorske (18)
- Tipska deskriptorska u C++ (19)
- Objekti i promjenjive (13)

3 → računanje

- Iskaz (8)
- Potpis funkcije: ime, broj parametara, tipovi parametara (ne i povratna vrijednost!) (18)
- C++ je statički tipiziran jezik - tip svih parametara zna se tokom prevođenja

4 → Greške

- Врсте грешака (2), Механизам изузетака (15, 16)
- Предуслови/очекивања (19) - представљају обавезна односно шта то функција очекује од аргумената
- Постуслови/гаранције (20) - шта ф-ја гарантује да ће дати тачно након неког задатка

6 → Писање калкулатора

- Појмови (објекат, прављење, типа)! (17) (слика)

7 → Функције

- Декларација (3) : мене и тип повр : користи
- Дефиниција (4) : направи
- Досег (9)
- Функције: Декларација и дефиниција (13)
- Прослеђивање параметара по вредности (14) (копија)

- Прослеђ
а такви
- Прослеђ
- (20) -
- Имена

- Разли
- Кон
- Стру
- Const
- Под
- Пре
* Да
нек

- Прослеђивање п. по референци: & у дефиницији, а позив нормалан (15)
- Прослеђивање п. по конст. референци (18): конверзија
- (20) - примјери! → црвено: не може!!!
- Стечени простор (23)

{ 9 → Index }

- Разлике приступа (30 и 31)
- Конструктори, иницијализација (...)
- Енумерације (36)
- Const (39)
- Подјела ф-ја (46)
- „Преклапање" операција (49)

* На начина да се дефинишу функционалности које одјекују неког типа може да има: конверзија и енумерације

9 → Улазно-излазни токови

10

- Код за фајлтеката
- Грешке улаза и излаза X
- Преклапање операције << и >> (29, 30)

- Манипулатори форматирања и прецизности X
- Уметнавање штампа vs уметнавање мењач (20 и 22)

11

12 - Организација графичких класа

- Домен примене / идеал правоугаоника (2) - да се концепти из домена примене изразе директно у коду
- Библиотека (3)
- Вектори (ограничења) - (5)
- Показивачи (6) - тип података који може садржати адресу неког објекта
- Статичка типизираност (8)

- Наслеђивање (9, 10)
- Полиморфизам (виртуелне ф-је) (13)

Битне се слике!!!

обичне
* ф-је не утичу на величину променљиве (док виртуелне утичу)
* Обичне функције: Када се ф-ја позива преко показивача (→), позвате се она ф-ја која одговара типу показивача!!! (Тип* ↖ = м)
* Виртуелне ф-је: Када се ф-ја позива преко показивача (→), не позива се на основу типа показивача, него на основу самог објекта!!!

12a - ... - тек од 12. слајда битно

- ООП = наслеђивање (основна, изведена класа) + полиморфизам (виртуелне ф-је) + енкапсулација (скривање података) (12)
- енкапсулација: public, protected, private
- прекривање виртуелне ф-је (15) (17)

Битна слика!!!

N Приликом прекривања виртуелних ф-ја, override се може писати али и не мора. Override се не може ти-

Kada nema šta da override-uje!

- „Mišo strela" (10) - klasa sa mišjom brzinom o.

13 - Virtuelne funkcije - ide nedeljno ovo

- using u typedef (8)

14 - Grafička korisnička strela

- GUI = OOP + Događaji (2)
- Widget = Pravougaonik koji prikazuje (11)
- Shape = Pravougaonik koji prikazuje
- Kontekst - Povezuje događaj sa nekom akcijom (14)
  (gde strela na ekranu)

15 - Pokazivači, nizovi i referencе

- Aritmetika bajtova / pokazivača - 3 tipa
- Niz (12)
- Pokazivači (10)

- Купрење меморије: шкрола кода програм сам не ослободи алоцирање меморије (bet се ослободи на крајот кодот на програма) (27)
- new (алоцирање), delete (ослободување) (24)
- (30) - Не постојат имплицитни конверзии помеѓу покажувачи, но постојат помеѓу типовите на кои покажуваат
- void* (38) - покажувач на некој објект кој тип е не знам

16 - Вектор

- Конструктор копија (14), Оператор доделе (16)
- Покажувачи vs Референци (26)

17 - Наставак

- Промена големина кеширање (дуплирање)
- explicit (12) → Алеа...
- Шаблони (30): template <typename T> class Vector
  - од 37 слајд не треба целосно...
- (35) ← може (малку)!!

template<class T>
class Vector

18 - Библиотека станд. шаблона - STL

- Алгоритми и контејнери: веза=итератори (27)
- Предикати (36), Функтор / функцијски објекти - ф-ја се имплементира као објекат класе (нпр. структура) у структури се дефинише bool operator() (парам...) const {...} (прима 1 параметар и враћа bool вредност)
- Ламбда израз / „функторски литерал" (50)
- iterator, const_iterator (57)
- vector: insert() и erase() (58)
- list: insert() и erase() (62)
- Итерирање кроз контејнере (64)

19 - Наставак STL

- Мапа (13)

```
for (const auto& p : words)
    cout << p.first << p.second;
```

20 - Матрица

- МУВ конструктор (конструктор преместване; Нема копиране, само се „прехвърля репрезентацията" (27)

  Matrix (Matrix && a) : ... {}

- МУВ деструктор (30)

- ако се у дефиницији функције налази &(адреса) у позиву иде нормална променљива: y

- ако се у дефиницији функције налази *y (показивач) у позиву иде &y (адреса)

* Обратити пажњу у задатку код наслеђивања класа да ли родитељска класа садржи виртуелну ф-ју или обичну ф-ју !!!

- Када имамо показивач на неки објекат (Tip* p = &V), тада се атрибутима тог показивача/објекта приступа преко → (p->a=5)

* const се може у не-const, а обрнуто може!!

- У позиву ф-је не може никад сам показивач!
- Конструктор копије: return *this
  и оператор доделе

Итератори

- Обратити пажњу на задатак на вежбама када је у питању **листа*** и када је **вектор***
- Одрадити овај задатак где се итерира кроз const листу или вектор? const_iterator

* За **листу** се са алгоритмима сјетамо да су подаци разбацани по меморији, па зато покретни елементи остају на истим меморијским локацијама, али не остају на истим позицијама (јер се мало који дрвиће или фотоаф).
* За **вектор** важи оно што смо рекли за низ на алгоритмима тј. да се чувају индексирањем, индекси претходних елемената остају исти, а касније не

Може брисање преко insert(it, 99), не само push_back()

претходни елем. на истим позицијама, остали се мијењају

erase()

- ~~Set (структура под)~~
- ~~Механизам чувања и полиморфизам~~
- Наћести дар о модификаторима приступа у C++

монитора